

```
In [3]: import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split, KFold
from sklearn.preprocessing import OneHotEncoder, StandardScaler, Multi
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.ensemble import RandomForestClassifier
from sklearn.multiclass import OneVsRestClassifier
from sklearn.metrics import accuracy_score

df = pd.read_csv("../data/filteredvehpub.csv") # adjust path as need
df.head()
df.shape
```

```
Out[3]: (256115, 14)
```

```
In [4]: import numpy as np
invalid_values = [-9, -8, -7, -88, 99, "99", "XX", "xx", "XX ", "-88",
df = df.replace(invalid_values, np.nan)
df = df.dropna()
```

```
In [5]: #df["VEHTYPE"] = df["VEHTYPE"].replace([5, 6], np.nan)
#df = df.dropna()
```

```
In [6]: counts = df["MAKE"].value_counts()
counts.describe()
```

```
Out[6]: count      53.000000
mean      4607.245283
std       8163.852477
min        58.000000
25%       286.000000
50%      1580.000000
75%      4639.000000
max      34768.000000
Name: count, dtype: float64
```

```
In [11]: df["MAKE"] = df["MAKE"].astype(str) # convert everything to string f
df["MAKE"] = df["MAKE"].str.strip() # remove whitespace
df["MAKE"] = df["MAKE"].astype(int) # convert to integer

counts = df["MAKE"].value_counts()
rare_makes = counts[counts < 1000].index

df["MAKE"] = df["MAKE"].where(~df["MAKE"].isin(rare_makes), 98)
df["MAKE"].value_counts()
```

Out[11]: MAKE

12	34768
49	33623
20	31188
37	23829
7	12545
35	11684
98	9164
2	7293
48	6932
23	6896
55	6765
59	4967
18	4852
6	4703
63	4639
30	4608
34	4536
41	4335
42	3642
72	3064
19	2664
22	2659
54	2536
51	1831
14	1798
13	1787
24	1580
32	1491
58	1416
52	1236
53	1153

Name: count, dtype: int64

```
In [13]: hh_makes = (  
    df.groupby("HOUSEID")["MAKE"]  
        .apply(lambda s: sorted(set(s)))    # unique, sorted list of make  
        .reset_index(name="MAKE_LIST")  
    )  
  
    hh_makes.head()
```

Out[13]:

	HOUSEID	MAKE_LIST
0	30000007	[19, 20, 49]
1	30000008	[20]
2	30000012	[12, 58]
3	30000019	[37, 98]
4	30000029	[20, 49]

In []:

In []:

In []:

In []:

In [17]:

```
household_feature_cols = [  
    "HOUSEID",  
    "HHSIZE",  
    "HHFAMINC",  
    "LIF_CYC",  
    "CENSUS_R",  
    "HH_RACE",  
    "HOMEOWN",  
    "WRKCOUNT",  
    "URBAN",  
    "URBANSIZE",  
    "DRVRCNT"  
]  
  
hh_feat = df[household_feature_cols].drop_duplicates("HOUSEID")  
  
hh = hh_feat.merge(hh_makes, on="HOUSEID")  
hh.head()
```

Out[17]:

	HOUSEID	HHSIZE	HHFAMINC	LIF_CYC	CENSUS_R	HH_RACE	HOMEOWN
0	30000007	3	7.0	10.0	3	2.0	1.0
1	30000008	2	8.0	2.0	2	1.0	1.0
2	30000012	1	10.0	1.0	1	1.0	1.0
3	30000019	2	3.0	2.0	3	1.0	1.0
4	30000029	2	5.0	10.0	2	1.0	1.0

In [19]:

```
feature_cols = [  

```

```

    "HHSIZE",
    "HHFAMINC",
    "LIF_CYC",
    "CENSUS_R",
    "HH_RACE",
    "HOMEOWN",
    "WRKCOUNT",
    "URBAN",
    "URBANSIZE",
    "DRVRCNT"
]

```

```

X = hh[feature_cols]
y_list = hh["MAKE_LIST"]  # list of strings per row

```

In [21]: **from** sklearn.preprocessing **import** MultiLabelBinarizer

```

mlb = MultiLabelBinarizer()
Y = mlb.fit_transform(y_list)

print("Classes:", mlb.classes_)
print("Y shape:", Y.shape)  # (n_households, n_makes)

```

```

Classes: [ 2  6  7 12 13 14 18 19 20 22 23 24 30 32 34 35 37 41 42 48 4
9 51 52 53
54 55 58 59 63 72 98]
Y shape: (118549, 31)

```

In [23]: `X_train, X_test, Y_train, Y_test = train_test_split(`
`X, Y,`
`test_size=0.2,`
`random_state=42`
`)`

In [25]: `numeric_features = ["HHSIZE", "HHFAMINC", "WRKCOUNT", "DRVRCNT"]`
`categorical_features = ["LIF_CYC", "CENSUS_R", "HH_RACE", "HOMEOWN", "`

In [48]: **from** sklearn.compose **import** ColumnTransformer
from sklearn.preprocessing **import** OneHotEncoder, StandardScaler
from sklearn.multiclass **import** OneVsRestClassifier
from sklearn.ensemble **import** RandomForestClassifier

```

preprocess = ColumnTransformer(
    transformers=[
        ("num", StandardScaler(), numeric_features),
        ("cat", OneHotEncoder(handle_unknown="ignore"), categorical_fe
    ]
)

base_rf = RandomForestClassifier(
    n_estimators=100,
    random_state=42,

```

```

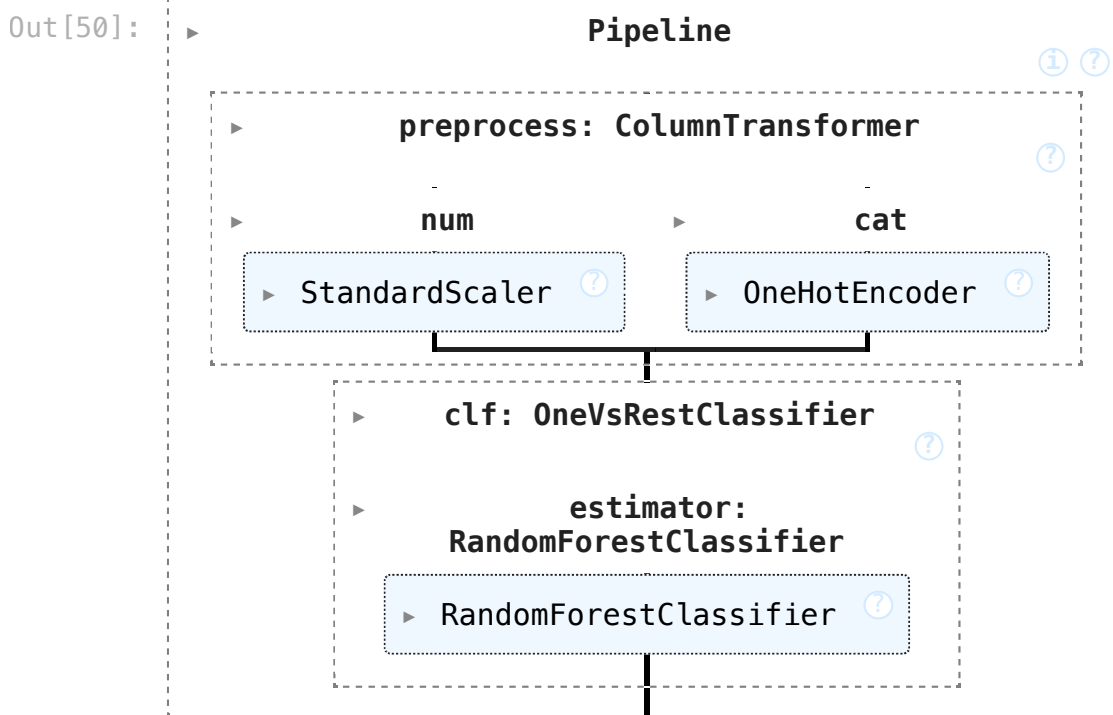
    n_jobs=-1,
    min_samples_split=4,
    min_samples_leaf=2
)

multi_rf = OneVsRestClassifier(base_rf, n_jobs=-1)

pipe = Pipeline([
    ("preprocess", preprocess),
    ("clf", multi_rf)
])

```

In [50]: `pipe.fit(X_train, Y_train)`



```

In [30]: # Access the fitted classifier and preprocessing
clf_step = pipe.named_steps["clf"]
X_test_trans = pipe.named_steps["preprocess"].transform(X_test)

# OneVsRestClassifier gives a list of estimators, each with predict_proba
probs_per_class = np.column_stack([
    est.predict_proba(X_test_trans)[:, 1] # P(has this make)
    for est in clf_step.estimators_
])

probs_per_class.shape # (n_samples, n_classes)

```

Out[30]: (23710, 31)

```

In [31]: top1_idx = np.argmax(probs_per_class, axis=1) # index of best make
top1_makes = mlb.classes_[top1_idx] # optional: actual

```

```
top1_makes[:10]
```

```
Out[31]: array([20, 20, 20, 12, 35, 37, 37, 12, 37, 20])
```

```
In [32]: correct_flags = []

for i in range(Y_test.shape[0]):
    # 1 if predicted make is actually one of the household's makes
    correct_flags.append(Y_test[i, top1_idx[i]] == 1)

top1_in_set_accuracy = np.mean(correct_flags)
print("Top-1-in-set accuracy:", top1_in_set_accuracy)
```

Top-1-in-set accuracy: 0.2618726275832982

```
In [33]: all_makes_flat = [m for makes in y_list for m in makes]
most_common_make = pd.Series(all_makes_flat).value_counts().idxmax()
print("Most common make:", most_common_make)
baseline_idx = np.where(mlb.classes_ == most_common_make)[0][0]
print("Index in ML-binarizer:", baseline_idx)
correct_flags = []

for i in range(Y_test.shape[0]):
    correct_flags.append(Y_test[i, baseline_idx] == 1)

baseline_top1_in_set = np.mean(correct_flags)
print("Baseline Top-1-in-set Accuracy:", baseline_top1_in_set)
```

Most common make: 12

Index in ML-binarizer: 3

Baseline Top-1-in-set Accuracy: 0.23390974272458878

```
In [ ]:
```